

1 Life-Cycle Model With Endogenous Labor Supply

- State variables:

- a , assets.
- η , transitory labor productivity shock.
- j , age.
- θ , permanent productivity type.

- Choice variables:

- a' , next-period assets.
- l , labor supply, or hours of work.

- Household optimization problem in recursive form:

$$V(a, \eta, j, \theta) = \max_{a', l} \{u(c, 1 - l) + \beta \psi_{j+1} E[V(a', \eta', j + 1, \theta) | \eta]\}.$$

The budget constraint is

$$c + a' = (1 + r)a + we_j \theta \eta l + pen_j,$$

with

$$a' \geq 0, \quad 0 \leq l \leq 1, \quad c \geq 0.$$

The policy functions are

$$\begin{aligned} a' &= g_a(a, \eta, j, \theta), \\ l &= g_l(a, \eta, j, \theta). \end{aligned}$$

Effective labor productivity is

$$h_j(\eta, \theta) = \begin{cases} e_j \theta \eta, & j < J_R, \\ 0, & j \geq J_R, \end{cases}$$

where e_j is the deterministic age-efficiency profile. In the Fortran code, $r = 0.04$, $w = 1$, $J = 80$, $J_R = 45$, $N_a = 200$, $N_\theta = 2$, and $N_\eta = 7$. The pension is

$$pen_j = \begin{cases} 0, & j < J_R, \\ 0.5 \cdot 0.33 \cdot \frac{1}{J_R - 1} \sum_{s=1}^{J_R-1} e_s, & j \geq J_R. \end{cases}$$

The period utility function is

$$u(c, 1 - l) = \frac{[c^\nu (1 - l)^{1-\nu}]^{1-\frac{1}{\gamma}}}{1 - \frac{1}{\gamma}},$$

with $\nu = 0.335$, $\gamma = 0.50$, and $\beta = 0.98$.

The transitory productivity process is discretized as an AR(1) process,

$$z' = \rho z + \varepsilon', \quad \varepsilon' \sim N(0, \sigma_\varepsilon^2),$$

with $\rho = 0.985$ and $\sigma_\varepsilon = 0.022$. The code stores $\eta = \exp(z)$ after discretization. The two permanent productivity types are

$$\theta_1 = \exp(-\sqrt{\sigma_\theta}), \quad \theta_2 = \exp(\sqrt{\sigma_\theta}),$$

with $\sigma_\theta = 0.242$ and equal type probabilities.

Initial condition: at age 1, all individuals start with zero assets and the middle transitory shock state, which corresponds to $z = 0$ and hence $\eta = 1$. The permanent productivity type is distributed uniformly across the two permanent types.

- Distribution:

$$\mu(a', \eta', j + 1, \theta) = \sum_a \sum_\eta I\{a' = g_a(a, \eta, j, \theta)\} \pi(\eta, \eta') \mu(a, \eta, j, \theta),$$

for $j = 1, \dots, J - 1$. If $g_a(a, \eta, j, \theta)$ is not a grid point, the code linearly redistributes mass across the two neighboring asset-grid points. Let $a_k \leq a' < a_{k+1}$ and choose ω such that

$$a' = \omega a_k + (1 - \omega) a_{k+1}.$$

Then the algorithm assigns the fraction ω of the mass to a_k and the fraction $1 - \omega$ to a_{k+1} .

The initial distribution is

$$\mu(a, \eta, 1, \theta_i) = \begin{cases} \pi_i^\theta, & a = 0, \eta = 1, \theta = \theta_i, \\ 0, & \text{otherwise.} \end{cases}$$